

Atividade lista 20 - Professor: Daniel Henrique Matos de Paiva
Aluna: Isabela da Silva Freitas

Questão 1:

Resposta: Na classe **Usuário**, deixar os atributos como **public** é perigoso porque qualquer parte do sistema pode alternar informações importantes sem nenhuma validação. Por exemplo, um usuário inadimplente poderia ter seu *statusAssinatura* alterado por “Ativa”, permitindo acesso ao conteúdo sem realizar o pagamento. Isso gera prejuízos financeiros para a empresa e compromete a segurança do sistema.

O conceito da Programação Orientada a Objetos que resolve esse problema é o **Encapsulamento**. Com ele, os atributos ficam definidos como **private**, impedindo o acesso direto por outras classes.

Os métodos Getters e Setters permitem controlar o acesso aos dados. Além disso, podem ser utilizados métodos de negócio para validar regras específicas antes de realizar alterações. Dessa forma, apenas operações autorizadas podem modificar o status da assinatura, protegendo a receita da empresa e garantindo maior segurança ao sistema.

Questão 2:

Resposta: Filmes e Séries possuem características em comum, como título, ano lançamento e gênero. Criar duas classes separadas faria com que esses atributos fossem repetidos, aumentando a quantidade de código e dificultando a manutenção.

Para resolver isso, podemos utilizar o conceito de **Herança**. Criamos uma superclasse chamada **Midia**, contendo os atributos e métodos comuns a todos os tipos de conteúdo.

```
public class Midia {  
    private String titulo;  
    private int anoLancamento;  
    private String genero;  
}
```

As classes **Filme** e **Serie** serão subclasses que herdam essas características utilizando a palavra-chave **extends**.

```
public class Filme extends Midia {  
    private int duracao;  
    private String diretor;  
}
```

```
public class Serie extends Midia {  
    private int temporada;  
    private int episodios  
}
```

Dessa forma, evitamos duplicação de código, facilitamos a manutenção e reduzimos o tempo necessário para corrigir erros ou implementar melhorias no sistema.

Questão 3:

Resposta:

A **Abstração** consiste em mostrar apenas o que é necessário para o usuário, escondendo os detalhes internos de funcionamento do sistema.

No caso da **DevFlix**, quando o cliente clica no botão “Dar Play”, ele vê apenas uma ação simples. Porém, internamente, o sistema executa várias tarefas, como verificar a assinatura, localizar o vídeo nos servidores, carregar o buffer e iniciar a transmissão.

Ao utilizar a abstração, toda essa complexidade fica escondida dentro de métodos específicos, permitindo que outras partes do sistema utilizem apenas a funcionalidade principal.

Isso é importante porque, se a equipe precisar atualizar o reprodutor de vídeo futuramente, poderá modificar apenas a implementação interna sem alterar a forma como o restante do sistema utiliza o recurso. Assim, o aplicativo continua funcionando normalmente, reduzindo riscos de falhas e facilitando a evolução do software. Além disso, o código fica mais organizado, compreensível e fácil de manter.

Boa parte da nota vem de relacionar os conceitos da POO com o negócio da empresa, e essas respostas já fazem essa ligação com segurança, receita e produtividade da equipe.